

CMSC 471 - Programming Assignment 1

A* Search and Heuristics with DOGCAT

Edmond Ndanji

AI use

I used Claude (Anthropic) to help write dc.py and these answers. I checked the code by running dcsolve.py and dctest.py against the sample outputs, checking that A* and UCS give the same costs, and checking that no heuristic ever went above the true optimal cost.

Question 1 Heuristic design and admissibility

$k(n)$ is the number of letters where n differs from the goal g . All three heuristics are 0 at the goal.

Steps

$$h(n) = k(n)$$

Each action fixes at most one wrong letter and costs 1, so at least $k(n)$ more cost is needed.

Scrabble

$$h(n) = \text{sum of value}(g[i]) \text{ for every position } i \text{ where } n[i] \neq g[i]$$

Every wrong position i has to get the letter $g[i]$ at some point, which costs $\text{value}(g[i])$. Other actions only add cost, so it never overestimates.

Frequency

$$h(n) = 0 \text{ if } n = g, \text{ otherwise } h(n) = (1 + \text{rarity}(g)) + (k(n) - 1) \times (1 + r_{\min})$$

$r_{\min}$ is the smallest rarity of any word of that length (0.0162 for 3 letters, 0.3515 for 4). At least $k(n)$ actions are needed. The last one enters g and costs $1 + \text{rarity}(g)$. Each other one enters some word and costs at least $1 + r_{\min}$.

Question 2 Does the heuristic actually help?

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.9 %
WHY → ASK	Scrabble	9	765	9	13	98.3 %
WHY → ASK	Frequency	38.855	808	38.855	634	21.5 %
BEAR → DUCK	Steps	4	691	4	6	99.1 %
BEAR → DUCK	Scrabble	11	2981	11	6	99.8 %
BEAR → DUCK	Frequency	38.240	1084	38.240	257	76.3 %

1. Same optimal cost?

Yes, in all six cases.

2. Largest reduction

Scrabble for both pairs.

3. Least informative

Frequency for both pairs.

4. Why

For steps and Scrabble, h is close to the real cost, since the best path mostly changes each wrong letter straight to the goal letter. Scrabble has the largest reduction because UCS expands more nodes with uneven letter costs, while A^* still goes almost straight to the goal.

For frequency, h charges every step except the last the smallest rarity in the dictionary, but real words on the path cost much more (beak is 11.46, ais is 13.89). So h is far below the real cost (7.27 vs 38.86 for WHY $\rightarrow$ ASK), and A^* acts more like UCS. WHY $\rightarrow$ ASK is worse because its best path takes 7 steps for only 3 wrong letters.

Question 3 Reversibility and direction-dependent costs

Every action is reversible, since both words are in the dictionary. That does not mean the reverse action costs the same.

Steps: yes

Every action costs 1 both ways, so a reversed path has the same cost.

Scrabble: no

The cost depends on the letter put in. cat $\rightarrow$ hat costs 4 (h), but hat $\rightarrow$ cat costs 3 (c). Also why $\rightarrow$ ask costs 9 but ask $\rightarrow$ why costs 14.

Frequency: no

The cost depends on the word entered, and the start word is never paid for. cat $\rightarrow$ hat costs $1 + \text{rarity}(\text{hat}) = 7.4457$, but hat $\rightarrow$ cat costs $1 + \text{rarity}(\text{cat}) = 7.8869$.

Question 4 LLM heuristic audit

1. Model, prompt and proposed heuristic

Model: Claude (Anthropic), claude-opus-5-5, in a fresh session with no access to my code. I added one line before the prompt: "Answer the question below using only your own reasoning. Do not use any tools, and do not read or run any files."

Prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By "more informative," we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Proposed heuristic: the last few words on any path must be close to the goal, so charge each of those steps the smallest rarity among words that close to the goal.

2. Formula

m_j = smallest rarity of any word that differs from g in at most j letters ($m_0 = \text{rarity}(g)$)

$h_R(n) = (1 + m_0) + (1 + m_1) + \dots + (1 + m_{d-1})$, where $d = h_0(n)$, and $h_R(g) = 0$

3. Is it admissible?

Yes. The word j steps before the goal can differ from g in at most j letters, so entering it costs at least $1 + m_j$. The last d steps of any path cost at least $h_R(n)$, and the path has at least d steps, so it never overestimates.

It is also at least h_0 everywhere, since every $m_i \geq 0$, and bigger at every non-goal word, since $\text{rarity}(g) > 0$ for every word. I also checked 86,196 word and goal pairs and it never went above the true cost.

4. Repair

None needed.

5. Test against h_0

Word pair	Heuristic	Solution cost	Nodes expanded
WHY → ASK	h_0	38.855	755
WHY → ASK	h_R	38.855	449
BEAR → DUCK	h_0	38.240	720
BEAR → DUCK	h_R	38.240	75

Same optimal costs, with 40.5 % fewer nodes on WHY → ASK and 89.6 % fewer on BEAR → DUCK. The evidence supports using h_R .